

Azure Kubernetes Agentic Ops

Project Solution Document

Safety-First Incident Agent for Azure Kubernetes Service

Architecture · Observability · Evaluation · Workflow

1. Executive Summary

Azure Kubernetes Agentic Ops is a safety-first, event-driven incident agent designed for Azure Kubernetes Service (AKS). The solution implements a controlled autonomy model in which safe, reversible conditions are remediated automatically, while all other cases are escalated with structured evidence for human review.

The current Minimum Viable Product (MVP) focuses on a single, high-value remediation path: correcting an image-name typo in an Azure Container Registry (ACR) Deployment (for example, `paymnets-api` → `payments-api`). This narrow scope enables rigorous safety guarantees while demonstrating the full agentic loop—detection, diagnosis, policy authorization, action, verification, and audit.

The broader platform vision organizes operations into three support layers—K8s Self-Cure, Agent Support, and Human Support—so that repetitive toil is removed from on-call shifts and human experts receive complete, grounded context only when judgment is required.

2. Business Problem and Goals

2.1 Problem Statement

Cluster, networking, storage, RBAC, scheduling, and configuration incidents force engineers to switch between `kubectl`, Prometheus, Grafana, Loki, documentation, and ticketing systems. The result is elevated mean time to recovery (MTTR), inconsistent responses, and significant on-call load.

2.2 Goals

- **Reduce median time-to-RCA** — from incident creation to a grounded diagnosis and recommendation, measured against the current manual baseline.
- **Re-validate quality with every release** — through automated evaluation gates on groundedness, safety, and diagnostic accuracy.
- **Preserve human authority** — outside a tightly defined whitelist of safe actions; the agent never invents cluster state.

2.3 Target Audience

Platform Engineering, SRE, Cloud Operations, on-call teams, and technical leads who need fast, reliable root-cause analysis.

3. Solution Overview

3.1 Three Support Layers

The platform operates through three coordinated layers:

Layer	Responsibility
K8s Self-Cure	Executes only whitelisted, safe, reversible actions without waiting for approval (in the MVP: fix_image under strict policy).
Agent Support	Read-only diagnostics: gathers source-backed facts from the cluster and correlating services; proposes actions treated as untrusted input.
Human Support	Approval, escalation, and higher-impact decisions. Receives full RCA, evidence sources, and recommended next steps.

3.2 Design Principles

- **Tools are the source of truth** — cluster state is never invented; every fact carries an explicit source.
- **Model output is untrusted** — the LLM may propose only a corrected image reference; SelfCurePolicy applies deterministic checks before any write.
- **Least privilege** — Workload Identity, namespace-scoped RBAC, allowlisted registries, and NetworkPolicy for the optional webhook.
- **Event-driven activation** — the system does not poll without purpose; it reacts to ImagePullBackOff / ErrImagePull (or webhook triggers).
- **Auditability** — every decision emits a structured audit event with correlation ID, status, reason, groundedness, and evidence sources.

3.3 MVP Scope

The MVP automatically corrects one narrowly defined condition: an image-name typo in one ACR Deployment. All other cases are escalated with collected evidence. Supported constraints include:

- One managed application namespace and one allowlisted ACR.
- Deployments only (Pod → ReplicaSet → Deployment ownership must be established).
- Exactly one container failing to pull its image.
- In-memory incident state and deduplication (single replica by design).

4. Agent Architecture

4.1 Component Model

The runtime is composed of modular components with clear contracts:

Component	Role
PodWatcher	Watches one managed namespace for ImagePullBackOff and ErrImagePull; raises IncidentTriggers.
AksDiagnosticProvider	Read-only collection of Pod status, Kubernetes Events, and previous logs; produces Evidence with source URIs.

FoundryDiagnosticProvider	Sends evidence to Azure AI Foundry / Azure OpenAI; receives validated JSON (groundedness, summary, optional safe_action).
SelfCurePolicy	Deterministic gate: authorizes fix_image only when all policy predicates hold; otherwise raises PolicyViolation.
AcrRegistry	Verifies image tag existence via ACR OAuth (AAD → refresh → repository-scoped pull token).
AksActionExecutor	Patches only the authorized container image; verifies healthy rollout before resolving the incident.
IncidentOrchestrator	Deduplicates by namespace/workload, sequences diagnostics → policy → action → verify, emits audit events.
Webhook API (optional)	POST /api/v1/incidents with token auth; verifies namespace scope and Pod ownership before handling.

4.2 Contracts and Data Model

Core types enforce a clear Facts → Recommendation → Approval flow:

- **IncidentTrigger** — namespace, pod, workload, reason, correlation_id; deduplication key is namespace/workload.
- **Evidence** — name, value, source (for example k8s://pod/ns/pod-name).
- **Facts** — evidence items, groundedness score [0,1], optional safe_action, summary, pull_failures.
- **ActionRequest** — name and parameters; for fix_image the policy binds namespace, deployment, container, old_image, new_image.
- **Incident** — status (open | resolved | escalated), facts, optional recommendation, reason, executed action.

4.3 Self-Cure Authorization Rules

fix_image is allowed only when all of the following hold:

1. Exactly one container has an image-pull failure.
2. The model supplies exactly one parameter: image.
3. The registry is allowlisted and unchanged between current and proposed image.
4. Repository and tag differ by at most two edits (optimal string alignment; transposition counts as one).
5. The current image is missing from ACR.
6. The proposed image exists in ACR.
7. The Deployment still carries the image observed during diagnosis (no concurrent change).

The model cannot choose namespace, workload, deployment, or container. Any policy, model, ACR, Kubernetes, or rollout error results in escalation.

4.4 Identity and RBAC

- Workload Identity (OIDC) binds the agent ServiceAccount to a user-assigned managed identity.
- Role is bound only in the managed namespace: get/list/watch pods; get logs and events; get ReplicaSets; get/patch Deployments.

- Identity holds Cognitive Services OpenAI User on the Foundry resource and AcrPull on the registry.
- The image-only restriction is enforced in application policy, not solely in Kubernetes RBAC.

5. Workflow

5.1 End-to-End Incident Flow (MVP)

1. PodWatcher observes ADDED/MODIFIED pod events in the managed namespace and detects ImagePullBackOff or ErrImagePull.
2. Ownership is resolved (Pod → ReplicaSet → Deployment). Pods not owned by a Deployment are skipped.
3. IncidentOrchestrator opens or deduplicates an incident (key: namespace/workload; cooldown applies).
4. AksDiagnosticProvider collects pod phase, container states, events, and pull-failure evidence with source URIs.
5. FoundryDiagnosticProvider sends evidence to the model (temperature 0, JSON response format) and parses groundedness, summary, and optional safe_action.
6. If groundedness is below threshold (default 0.85) or no safe action is proposed, the incident is escalated.
7. SelfCurePolicy authorizes the action or raises PolicyViolation / OutOfScope.
8. AksActionExecutor patches the container image and polls until the Deployment is fully rolled out and available, or times out.
9. Incident is marked RESOLVED only after a healthy rollout; otherwise ESCALATED with reason and evidence.
10. A structured audit log entry is emitted for every decision.

5.2 Escalation Criteria

Escalation occurs when any of the following is true:

- Groundedness is below the configured quality threshold.
- No safe action is proposed by the model.
- Policy predicates fail (wrong registry, excessive edit distance, image still present in ACR, multiple pull failures, etc.).
- Action executes but the workload does not become healthy within the verification timeout.
- Any exception during diagnostics, registry access, or Kubernetes API interaction.

5.3 Optional Webhook Path

When AGENTIC_OPS_WEBHOOK_TOKEN is configured, POST /api/v1/incidents accepts external triggers. Every request must present X-Webhook-Token. The service verifies that the namespace is managed and that the submitted workload matches the Deployment that owns the Pod. The Service is cluster-internal; NetworkPolicy restricts ingress to namespaces labelled agentic-ops/webhook-sender=true.

5.4 Target Vision Workflow (Beyond MVP)

The project concept extends the same pattern to broader triggers (for example CrashLoopBackOff) and a larger whitelist of reversible actions (rollout restart, scale, single-pod eviction), with automatic rollback if recovery fails. Outside the whitelist, Agent Support produces recommendations and Human Support owns execution.

6. Observability Strategy

6.1 Design Intent

Observability is a first-class concern: every incident path must be reconstructible, measurable, and cost-aware. The MVP emits structured audit events; the target architecture adds full tracing, metrics, and cost governance.

6.2 Traces

- Complete incident path: trigger → tool calls → model inference → policy decision → action → verification.
- Correlation ID propagated from watcher or webhook through audit logs.
- Duration of every step and kubectl / API call; tool status and token/cost per model call.

6.3 Metrics

- Support-layer latency (p50 / p95 / p99).
- Tool success and error rates.
- Groundedness score distribution (facts ↔ sources).
- Time-to-RCA and time-to-resolution for self-cured vs. escalated incidents.
- Operator feedback (when available) for recommendation quality.

6.4 Cost Governance

- Token budget per ticket with alerts for runaway queries.
- Caching and rate-limiting of expensive tool calls before they reach the cluster or the model.
- Low temperature and structured JSON outputs to improve consistency and reduce retries.

6.5 Audit Log (MVP)

Each decision writes a JSON audit record including: event type, correlation_id, deduplication key, trigger reason, status, reason, groundedness, summary, action name and parameters, action_executed flag, and evidence_sources.

7. Evaluation Approach

7.1 Test Datasets

- **Golden** — realistic operational incidents (image typo, known-good and known-bad outcomes).
- **Synthetic** — failure variants (hallucinated tags, existing-but-unpullable images, multi-container pods, out-of-scope namespaces).

- **Live** — anonymized production-like signals (target state).
- **Adversarial** — attempts to trigger unsafe actions or inject instructions via evidence strings.

7.2 Evaluation Criteria

Criterion	Description
Diagnostic accuracy	Correct identification of root cause from evidence.
Groundedness	Conclusions follow from tool-backed facts; quality gate ≥ 0.85 .
Role adherence	Model proposes only allowed actions and parameters.
Safety	No unauthorized write; policy blocks all out-of-contract proposals (target: 100%).
RCA quality	Summary and evidence sources are useful for human operators.
Tool usage	Appropriate, minimal tool calls; no fabricated cluster state.
Consistency	Stable outcomes under low temperature and prompt versioning.
Time-to-insight	Latency from trigger to resolution or escalation.

7.3 Integration and Quality Gates

- Automated evaluation in CI/CD with a 60-second per-test timeout.
- Quality gates: groundedness ≥ 0.85 ; safety = 100% on adversarial and policy suites.
- Unit and end-to-end tests cover policy authorization, orchestrator paths, ACR client behavior, Foundry response parsing, watcher, and webhook authentication.
- Weekly review of worst incidents and A/B tests of new agent or prompt versions (target process).

7.4 Governance and Safety Controls

- **Consistency** — low temperature, cached diagnostic results, prompt versioning, structured JSON between layers.
- **Safety** — guardrails block write/exec outside the whitelist; read-oriented RBAC; human-in-the-loop for non-whitelisted changes; complete audit log.
- **Maintainability** — modular layers and tools, feature flags, regression tests, documented Facts → Recommendation → Approval contracts.
- **Grounding** — Agent Support never invents cluster state; tools are the source of truth; recommendations rely only on facts and the knowledge base.

8. Deployment Model

8.1 Runtime Placement

The agent runs as a single-replica Deployment in the agentic-ops namespace and manages one application namespace. Recreate strategy avoids dual-pod races during updates. Incident state is in-memory by design for the MVP.

8.2 Prerequisites

- AKS with OIDC issuer and Workload Identity enabled.
- Azure Container Registry reachable from the cluster.
- Azure AI Foundry or Azure OpenAI-compatible endpoint with a deployed model.
- Azure CLI and kubectl for bootstrap.

8.3 Build and Ship

The container image is built with Python 3.12-slim, non-root user, and pinned dependencies via constraints.txt. Image is built into ACR (az acr build) and deployed via rendered manifests that substitute identity, Foundry endpoint, deployment name, image reference, managed namespace, and ACR login server.

9. MVP Limitations and Roadmap

9.1 Current Limitations

- Only image-typo remediation is automatic.
- One namespace, one ACR, Deployments, and one failing container per Pod.
- In-memory state; single replica.
- Tests use fake Kubernetes, ACR, and Foundry endpoints—identity, NetworkPolicy, and live rollout behavior must be validated in a non-production AKS environment.
- Model groundedness is self-reported; registry and policy checks provide the effective safeguards.

9.2 Intended Evolution

- Expand the self-cure whitelist with reversible actions (restart, scale, eviction) and automatic rollback.
- Additional triggers (CrashLoopBackOff and alert-driven paths).
- Persistent incident store and multi-replica coordination.
- Full OpenTelemetry traces and metrics export.
- Continuous evaluation pipelines with golden/synthetic/adversarial suites in CI.
- Human approval workflows for non-whitelisted recommendations.

10. Conclusion

Azure Kubernetes Agentic Ops demonstrates a practical, safety-first approach to agentic operations on AKS. By constraining autonomy to a rigorously checked remediation path and treating model output as untrusted input, the system reduces toil on a common failure mode while preserving auditability and human control.

The architecture—event-driven triggers, source-backed diagnostics, deterministic policy, verified execution, and structured evaluation—provides a foundation that can grow from a narrow MVP into a broader multi-layer operating model without sacrificing the principle that tools, not models, are the source of truth for cluster state.